

Reviewer Pack — Minimal Order of Eichler-Shimura Hecke Operators and DPLL Se...

Entry 822dc076d3d4 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-11 10:50:05 UTC

Minimal Order of Eichler-Shimura Hecke Operators and DPLL Search Tree Depth Correlation

Entry ID: 822dc076d3d4 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-11 10:50:05 UTC

1. Conjecture

Field A (mathematical branch): Eichler-Shimura Theory **Field B** (complexity object): Boolean Satisfiability Problem - DPLL Search Tree

Statement:

For every Boolean formula φ with n variables, the minimal order of Eichler-Shimura Hecke operators associated with the automorphism group of the corresponding matroid is linearly correlated with its DPLL search tree depth $d(\varphi)$, such that $\log(\min_order) = \Theta(d(\varphi))$.

Rationale (proposer's reasoning):

Eichler-Shimura Theory provides a rich structure for studying arithmetic properties of modular forms, which may encode information about computational structures in Boolean satisfiability problems. Minimal order of Hecke operators could potentially reflect the complexity of DPLL search, offering a new perspective on the proof complexity of SAT.

Taxonomy category: EICHLER_SHIMURA_THEORY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): e85242fc01beb12a

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if and only if for all Boolean formulas φ with n variables ($n \leq 40$), the correlation coefficient between $\log(\min_order)$ of Eichler-Shimura Hecke operators and DPLL search tree depth $d(\varphi)$ across 30 random seeds is ≥ 0.7 , where $\min_order$ is the minimal order of the Hecke operators and $d(\varphi)$ is the DPLL search tree depth.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_formula(n):
    if n == 1:
        return 'x'
    else:
        left = generate_formula(random.randint(1, n-1))
        right = generate_formula(n - len(left.split('&')) - len(right.split('|')))
        op = random.choice(['&', '|'])
        return f'({left}{op}{right})'

def dpll_search_tree_depth(formula):
    if formula == 'x':
        return 0
    elif '&' in formula:
        left, right = formula[1:-1].split('&')
        return max(dpll_search_tree_depth(left), dpll_search_tree_depth(right)) + 1
    else:
        left, right = formula[1:-1].split('|')
        return max(dpll_search_tree_depth(left), dpll_search_tree_depth(right)) + 1

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        formula = generate_formula(n)
        min_order = len(formula.split('&')) + len(formula.split('|'))
        d_phi = dpll_search_tree_depth(formula)

        if d_phi == 0:
            continue
```

```

log_min_order = math.log(min_order) if min_order > 0 else -math.inf
results.append((log_min_order, d_phi))

if not results:
    return {
        "metric_name": "correlation_coefficient",
        "metric_value": None,
        "instances_tested": 0,
        "n_max": max(n_values),
        "conjecture_holds": False,
        "counterexample": "empty_formula"
    }

n = len(results)
sum_log_min_order = sum(x[0] for x in results)
sum_d_phi = sum(x[1] for x in results)
sum_log_min_order_squared = sum(x[0]**2 for x in results)
sum_d_phi_squared = sum(x[1]**2 for x in results)
sum_log_min_order_d_phi = sum(x[0] * x[1] for x in results)

mean_log_min_order = sum_log_min_order / n
mean_d_phi = sum_d_phi / n

cov = (sum_log_min_order_d_phi - mean_log_min_order * mean_d_phi) / (n - 1)
var_log_min_order = (sum_log_min_order_squared - mean_log_min_order**2) / (n - 1)
var_d_phi = (sum_d_phi_squared - mean_d_phi**2) / (n - 1)

std_dev_log_min_order = math.sqrt(var_log_min_order)
std_dev_d_phi = math.sqrt(var_d_phi)

correlation_coefficient = cov / (std_dev_log_min_order * std_dev_d_phi)

return {
    "metric_name": "correlation_coefficient",
    "metric_value": correlation_coefficient,
    "instances_tested": n,
    "n_max": max(n_values),
    "conjecture_holds": abs(correlation_coefficient) >= 0.7,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or list(range(2, 31)) # Default to first 30 primes if

    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_value\": {result['metric_value']}\"}}")
        results.append(result)

    mean_value = sum(x['metric_value'] for x in results if x['metric_value'] is not None) / len(results)
    std_dev_value = math.sqrt(sum((x['metric_value'] - mean_value)**2 for x in results if x['metric_value'] is not None) / (len(results) - 1))
    support_fraction = sum(1 for x in results if x['conjecture_holds']) / len(results)

```

```

if all(x['conjecture_holds'] for x in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_dev_value} support_fraction={support_fraction}")
elif any(not x['conjecture_holds'] for x in results) and support_fraction >= 0.8:
    print(f"RESULT: FALSIFIED counterexample=\"correlation_coefficient < 0.7\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE insufficient_data_or_low_support_fraction")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_24b3e10f.py", line 99, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_24b3e10f.py", line 44, in run_trial
    formula = generate_formula(n)
              ^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_24b3e10f.py", line 22, in generate_formula
    left = generate_formula(random.randint(1, n-1))
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_24b3e10f.py", line 22, in generate_formula
    left = generate_formula(random.randint(1, n-1))
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_24b3e10f.py", line 22, in generate_formula
    left = generate_formula(random.randint(1, n-1))
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_24b3e10f.py", line 23, in generate_formula
    right = generate_formula(n - len(left.split('&')) - len(right.split('|')))
           ^^^^^
UnboundLocalError: cannot access local variable 'right' where it is not associated with a value

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it did not complete its execution to provide a correlation coefficient or any other relevant metric. | next: Re-run the test with proper error handling and ensure that it completes without crashing. If the test passes and meets the pre-registered support conditions, then re-evaluate the conjecture as SUPPORTED.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	18093
2	propose	ollama_remote	glm4:latest	0	0	12507
3	preregistration	ollama_remote	glm4:latest	0	0	9877
4	novelty	ollama_remote	glm4:latest	0	0	8523
5	novelty	ollama_remote	glm4:latest	0	0	8576
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14707
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18069
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9350
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13175
10	judge	ollama_remote	glm4:latest	0	0	11848

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 124724 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/822dc076d3d4.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/822dc076d3d4.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/822dc076d3d4.tar.gz` (if generated)